Tasklet Scheduling for Pipeline-Parallel LLM Training on Heterogeneous Networks

Dhaarun Abhimanyu S (Roll No: 23N216)
Karthick J S (Roll No: 23N221)
Mohana Kumar P (Roll No: 23N228)
Sanjeev M S (Roll No: 23N248)
Surya Narayanaa N T (Roll No: 23N256)

23N711 - PROJECT WORK - I

report submitted in partial fulfillment of the requirement for the award of degree of

BACHELOR OF ENGINEERING

Branch: COMPUTER SCIENCE AND ENGINEERING (AI&ML)

Of Anna University



October 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PSG COLLEGE OF TECHNOLOGY

(Autonomous Institution)

COIMBATORE – 641 004

PSG COLLEGE OF TECHNOLOGY

(Autonomous Institution)

COIMBATORE – 641 004

Tasklet Scheduling for Pipeline-Parallel LLM Training on Heterogeneous Networks

Bona fide record of work done by

Dhaarun Abhimanyu S (Roll No: 23N216)
Karthick J S (Roll No: 23N221)
Mohana Kumar P (Roll No: 23N228)
Sanjeev M S (Roll No: 23N248)
Surya Narayanaa N T (Roll No: 23N256)

Project report submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

Branch: COMPUTER SCIENCE AND ENGINEERING(AI&ML)

of Anna University

October 2026

……	……
Dr. L. S. Jayashree	**Dr. G. R. Karpagam**
Faculty Guide	Head of the Department

Certified that the candidates were reviewed on …….

……	……
Internal Examiner	Panel Members

ACKNOWLEDGEMENT

We express our heartfelt gratitude to our Principal (FAC), Dr. G. Thilagavathi, for providing us with this opportunity to develop and complete our project in our field of study

We would like to extend our sincere thanks to our Head of the Department, Dr. G. R. Karpagam, for encouraging and allowing us to present the project at our department premises for the partial fulfillment of the requirements leading to the award of BE degree.

We express our sincere gratitude to Dr. L. S. Jayashree, Department of Computer Science and Engineering, PSG College of Technology, for her guidance and support for the project "Tasklet Scheduling for Pipeline-Parallel LLM Training on Heterogeneous Networks".

We thank the Department of Computer Science and Engineering and PSG College of Technology for the academic environment in which this project is being developed. We also acknowledge our team's contributions to baseline reproduction, device support, scheduling, measurements and repeatable experimental workflows.

A sincere thanks to all the faculty members and staff of the CSE department for offering us the required support during the course of the project

SYNOPSIS

Training and fine-tuning large language models require computing resources that may not be available on a single machine. This project explores the use of underutilized laptops and workstations as a distributed training cluster, despite differences in their computing power, memory and network connectivity. It builds on the open-source Distributed Training of Foundation Models (DT-FM) framework [1], [4] and extends it with resource-aware allocation and dynamic scheduling of pipeline stages.

The framework profiles available devices and network connections to guide the allocation of model layers across participating ranks. It supports CPU and GPU device selection, directed network probes, manifest-based execution and transactional migration of layers between ranks [5]. Top-K gradient compression reduces the amount of data exchanged between data-parallel groups, following the principle of communicating selected gradient values [3]. MLflow, automated reporting, Docker packaging and continuous integration support monitoring and reproducibility [5]–[7].

The system is evaluated using BERT for question answering and mT5-small for Tamil text summarization. The mT5 evaluation uses the XL-Sum dataset [8]. The evaluation considers training time, communication overhead, GPU memory use and model quality. The reported results indicate that gradient compression reduces communication overhead and that distributed fine-tuning improves the evaluated model-quality metrics [9].

The project demonstrates an approach to training foundation models using heterogeneous machines, while recognizing the challenges posed by network communication and pipeline synchronization. The current framework requires fixed membership, so a node failure can stop training, although checkpoints may be available. Further work is needed to validate migration with more complex optimizer states and to assess the effect of gradient compression on model quality through matched experiments.

Keywords: Distributed Training, Pipeline Parallelism, DT-FM, Heterogeneous Computing, Resource-Aware Scheduling, Layer Migration, Gradient Compression, MLflow

CONTENTS

CHAPTER **Page No.**

Acknowledgement **(i)**

Synopsis **(ii)**

1. INTRODUCTION **1**

1.1. Background and Context 1

1.2. Need for Heterogeneous Training 2

1.3. Motivation 2

1.4. Problem Statement 3

1.5. Intended Audience 3

1.6. Objectives 3

2. LITERATURE STUDY **5**

2.1. Existing Solution Categories 5

2.2. Identified Gap 7

2.3. Relevance of Existing Research to DT-FM 8

3. SYSTEM ANALYSIS **9**

3.1. Feasibility Analysis 9

3.2. Requirements 10

3.3. Mitigation Strategy 13

4. SYSTEM DESIGN **15**

4.1. Architectural Overview 15

4.2. Use Case Diagram 17

CHAPTER **Page No.**

4.3 Sequence Diagram 18
4.4. Workflow Diagram 19
4.5. Deployment Design 20

5. SYSTEM IMPLEMENTATION 21
.....
5.1. Implementation Plan 21
5.2. Data Retrieval and Preprocessing 22
5.3. Communication Probing and Rank Ordering 22
5.4. Compute Profiling and Static Allocation 23
5.5. Live Resource Discovery and Dynamic Allocation 23
5.6. Transactional Migration and Rebalancing 24
5.7. Metrics, Logging and MLflow 24
5.8. Docker Deployment and CI 25
5.9. Testing and Validation 25

6. EXPERIMENTAL RESULTS AND DISCUSSION 26
.....
6.1. Evaluation Methodology 26
6.2. Distributed Training Performance and Resource Utilization 27
6.3. Dataset and model details 29
6.4. Model Convergence and Task Quality 30
6.5. Limitations 32
6.6. Analysis and Discussion of Experimental Results 33
6.7 Comparison Between Various Node sizes 35

7. CONCLUSION 36
.....
BIBLIOGRAPHY 37
.....
APPENDICES 39
.....

CHAPTER 1

INTRODUCTION

Training or finetuning a Large Language model today requires a lot of computing resources, especially when training a bigger model it requires us to rent high performance infrastructure from the cloud, making it difficult for students and scholars to acquire and experiment without that infrastructure. So there we thought we have consumer laptops of students and laboratory workstations which are underused since we cannot fit a larger model in a single one of them making them unusable.

Therefore we plan on making a Distributed Training Approach over a heterogeneous network via pipeline parallelism which physically splits the model, so therefore a single system doesn't want to hold the entire model.

1.1 Background and Context

Training a LLM today requires high performance compute infrastructure, which is costlier making it difficult for scholars and students to get access to this and do experiments in this domain, while there are consumer grade laptops of students and also laboratory workstations which remains underused since bigger model doesn't fit in the memory requirements for these systems. Cloud systems has larger number of compute infrastructure that is connected over a high bandwidth links which makes the distributed training of model faster and feasible In cloud environments. While planning to train model distributed in heterogeneous setup can help the students and scholars to poll their existing infrastructure and work with them , but it introduces latency and bandwidth issues, and providing fault tolerance in this setup can be tedious and efficiency of the training can be poor, which may sometimes be bottlenecked by the network links among the connected peers.

There are Three types of parallelism that is used In this domain setting, pipeline parallelism, which physically split the model into different pipeline stages and data parallelism, which trains different batches of training data to exploit parallelism and make the process faster. Here batches are split into microbatches

which pass through the stages during forward computation, and gradients return during backward computation.

1.2 Need for Heterogeneous Training

. Along with above stated issues we also noticed that devices which are polled may have different gpu drivers and compute capacity, assigning identical tasks to all of them , can make the process slower and tedious, there we must design a system which profile the nodes in the network and allocated the task based on the resources.

1.3 Motivation

1.3.1 Access to Underused Resources

First we saw whether there are students and scholars who can poll consumer grade gpu laptops and laboratory workstations that can support our distributed training experiment. Many scholars has accessible but smaller ones which on a single one cannot fit the entire model, but can be combined to hold one.

1.3.2 Hardware Portability

Existing implementations and studies don't support multiple gpu drivers supports , and therefore provides with the bottleneck of using the same drivers, we planned to use a common collective gpu backend so it is possible to run on heterogeneous devices

1.3.3 Reproducible Measurements

Existing implements doesn't support experimentation functions for scholars, the training run should be efficient and reproducible and each devices logs and models should Be collected, there are existing open - source software such as MLflow , integrating these softwares with this systems helps the scholars to make the experimentation reproducible

When speaking about experimentation, there are four main measurements that should be added for our distributed training run : **training window, process wall time, synchronization, checkpointing and failure status**.

1.3.4 Unequal Network Conditions

The latency and the bandwidth of the network links varies due to training of gpu's in a crowdsourced platform. Directed communication probes as inputs and missing links were kept explicitly in our pipeline ordering. Assuming that these probe values are ranking proxies, they do not measure one-way network capacity nor specify the entire training time.

1.4 Problem Statement

The project will tackle two interrelated systems problems: Design and implement a coherent model partition between a set of machines with varying compute, memory and network resources including uniform workload assignments and uniform rank assignments.

Transfer an ownership of a layer from one live to another without also transferring the supported training state; and consider any performance improvements separately from functionality. There is an ongoing proposal for auto-recovery worker loss.

1.5 Intended Audience

Students and project teams on parallelism on pipelines and heterogeneous computers.

Laboratory coordinators, who set up experiments using two- or three-computers that can be replicated.

Research staff who analyse the allocation of resources, the measures of communication and the state of continuity.

Developers who duplicate CPU/Gloo execution, or test an advertised vendor-specific device path.

1.6 Objectives

- Re-construct the same training path using local readily available pieces of equipment.
- CPU-staged communication to support the pipeline stages on various verified compute devices.

- Allow for measurement of communication, calculation of behavior and available memory.
- Create homogeneous rank and contiguous capacity-constrained layer assignments. Make dynamic starts or stop allocations at being started and optionally at optimized step completion boundaries.
- Transfer live ranks while maintaining weights, optimizer state and RNG state. Measure per rank; track experiments; generate lab comparison reports, and portable deployment workflows.

CHAPTER 2

LITERATURE STUDY

Among the three studies that are cited in the report - DT-FM[1] is used as the basis for the main implementation, ReCycle[2] for resilient training of deep neural networks, and Deep Gradient Compression[3] for improving communication optimization in networks, which are explained in the following section.

2.1 Existing Solution Categories

The literature and implementation is grouped into four different groupings such as

- Decentralized training with communication-aware configuration - DT-FM.
- Resilient pipeline adaptation of deep neural models - ReCycle.
- Sparse gradient communication - Deep Gradient Compression.
- Our portable extension of the DT-FM framework.

All these implementations solve different parts of the same problem - that is, training large neural networks across distributed systems.

DT-FM: Decentralized Training

Yuan et al. studied decentralized foundation-model training across heterogeneous networks. This paper mainly included the pipelining of nodes across different data parallel groups, along with scheduling experiments on those networks. Our solution extends this DTFM by adding portable execution with docker, resource allocation, measurement tools & dashboards(like MLflow), and validation workflows

ReCycle: Resilient Pipeline Adaptation

The ReCycle paper mainly tries to avoid training disruption (in case of node failures) by redirecting microbatches to the available data parallel groups(similar to ones in DTFM). That is, it dynamically uses idle time in the pipeline and reroutes tasklets. This paper is the basis for the proposed approach(but the current implementation uses fixed set of workers and transfers at present)

Deep Gradient Compression

The Deep Gradient Compression paper discusses about preserving the training behaviour while sending only a fraction of the gradients. To understand how gradients move across a network when model training:

- During the forward pass, the activations of outputs are computed and sent forward stage by stage
- During the backward pass, the gradients are computed and propagated backwards for updating weights at each stage

This gradient synchronization that happens across the data parallel groups during a backward pass is encouraged to be minimized, from the inference made when comparing gradient passes in our solution.

Futhermore, our implementation extension also enables metrics and memory allocation to be directly associated with the ranks(of nodes). This helps with repeatable device selection(as ranks essentially virtualize on top of the physical nodes), and configuration and maintenance of the devices.

Comparative Summary

Study / System	Source	Mechanism	Project Relationship
DT-FM	Yuan et al., 2022 [1]	Decentralized pipeline/data parallelism	Inherited research baseline
ReCycle	Gandhi et al., 2024 [2]	Microbatch rerouting and pipeline adaptation	Future recovery direction
Deep Gradient Compression	Lin et al., 2018 [3]	Sparse gradient exchange	Future communication direction
Team Extension	DT-FM project [5]	Portable runtime, allocation and migration	Local prototype

2.2 Identified Gap

Our work first extends from The open source repository , we identified that base paper contains only static scheduling and lacked fault tolerance which is crucial aspect for a real time distributed systems, and it also lacked support to multiple gpu drivers, since we have different gpu ecosystems , the need of heterogeneous driver support is important. After our baseline work was done, during our test run we found a bottleneck at gradient synchronisation for data parallel groups which is due to the larger model gradients sizes to be transferred to all peers, therefore there is a need to compress those gradients before the synchronisation step.

Our project works on these gaps by including support for multiple drivers through a common backend for tensor processing and docker image for environment driver setups. We also have referred a paper **ReCycle: Resilient Pipeline Adaptation [2]** for fault tolerance setup and implemented as part of our project

2.3 Relevance of Existing Research to DT-FM

The open source work distributed training on foundation models (DT - FM), first setup the base code for us, giving a baseline for static scheduling , we have forked the repository with proper licensing and extending the functionality over it . Beyond the current fixed-membership pipeline, implementing it requires the ability to recover from a trained state and the ability to have redundant peers, consistent optimizer/data progress or a clearly defined failure protocol, and a reasonable implementation of those requirements will require quite a bit of work. Deep Gradient Compression motivates measuring and reducing gradient traffic. There must be matched experiments (with enabled/disabled experiments); counts of the number of bytes; runtime monitoring and convergence/quality checks for benefits in compression.

When combined these studies lead a staged development route (1) to verify execution and (2) to measure resources and (3) to integrate allocation and (4) preserve live state, (5) assess and evaluate physical-host performance, and (6) check the other resilience mechanisms.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Feasibility Analysis

3.1.1 Technical Feasibility

The framework divides the model into pipeline stages and distributes them across the participating devices. Each device computes its assigned stage for a microbatch. Activations and gradients are transferred between stages through the CPU using the Gloo communication backend. This approach allows devices with different accelerators to participate without requiring a shared vendor-specific GPU communication backend. However, copying tensors between the GPU and CPU can add overhead and reduce throughput.

The framework also supports resource-aware allocation of model layers and the migration of supported training state between active ranks. These capabilities allow the workload to be adjusted during a training run when rebalancing is enabled. State continuity has been tested for FP32 SGD, while other optimizer states require further validation.

The current implementation uses a fixed set of workers. If a worker fails during training, the run can stop, although checkpoints may be available for restarting it. Automatic failure detection, reassignment of work from failed devices and recovery from checkpoints are proposed for future work.

3.1.2 Operational Feasibility

Our project has setup scripts with integrated ML flow and web page for training on the collected gpu devices, ml flow shows the per device logs and metrics to make the experiments repeatable on supported environments.

Tailscale is used for NAT Traversal which should be done since the gpu are present in different networks. The operators must agree on code/runtime versions,

model dimensions, vocabulary, batch settings, rank membership and network interfaces.

For local setup and working, There are seperate docker image and packages for cpu as well as gpu cuda version as well. Docker images make sure that the application is easily setup and reproducible in all the devices in the network

3.1.3 Economic Feasibility

Since we are using existing consumer gpu's and laboratory machines and avoiding dedicated uniform clusters it is very economically feasible. Open-source training, tracking and deployment tools support the prototype, but electricity, network administration, storage and setup time still matter.

3.2 Requirements

Requirements are divided into functional behavior, reproducibility and resource constraints.

3.2.1 Functional Requirements

The prototype must support the following functions:

1. Choose one of the supported local compute devices and report one of the compute device availability explicit choices.
2. Rank Initialization of model,batch and transport parameters.
3. Keep track of failed observations and measure the network link.
4. Continuous layers of transformers in given or found resources are used.
5. Calculate the forward/backward computation for microbatches and optimize steps between them.
6. For both successful and failure jobs document configuration, steps taken, timing and plan.

3.2.2 Non-Functional Requirements

The following are non-functional requirements:

1. **Reproducibility**: Integrated ML flow for model saving and proper documentation , each node logs are captured and documented along with the model artifacts, so every training run is reproducible
2. **Observability**: Identify stuck stages, unavailable network links ,unfinished tasks through structured logging per node.

3. **Portability**: Give the lowest baseline of CPU as well as explicit the bounds for vendor-specific paths.
4. **Measurement integrity**: Measurement of Warmup, steady state measurements, overhead and quality evaluation.

3.2.3 Hardware Requirements

1. Portable Experimental Baseline
   - The project supports CPU hosts that run on selected PyTorch environments
   - Memory that are needed for storing and training parameters and gradients, optimizer state, and activations.
   - Persistent data, caches and artifact storage.
2. Optional Accelerator Paths
   - NVIDIA hardware and supported CUDA runtime for the CUDA path documented.
   - AMD hardware / recorded ROCm environment, or supported Intel XPU hardware.
   - The recorded path for Intel UHD DirectML is in a separate Python 3.11 environment in WSL.
   - The memory used for installed accelerators is not regarded as memory available.

3.2.4 Software Requirements

Category	Tool / Technology	Purpose
CPU environment	Python 3.10; PyTorch 2.8.0	Published Docker baseline
Distributed transport	Gloo; CPU staging	Activation/gradient exchange
CUDA target	CUDA 12.6.3; CuPy 13.6.0	Image build; host driver required

Category	Tool / Technology	Purpose
CPU environment	Python 3.10; PyTorch 2.8.0	Published Docker baseline
Vendor paths	ROCm, XPU, DirectML	Separate native environments
Tracking	JSON; optional MLflow	Per-rank metrics and artifacts
Deployment	Docker; Compose; GitHub Actions	Reproducible packaging and CPU CI
Workloads	Synthetic; QQP; ongoing mT5/XL-Sum	System checks; separate application path

3.2.5 Interface Requirements

Rank commands must expose host/device selection, topology, model dimensions and batch settings.

- The launch manifest should be able to maintain identities measured in rank even when reordered, and it should contain executable commands.
- Record of progress should specify the current forward/backward or validation phase to enable operators to find stalls.
- A tracking interface should connect to other rank runs, but should not mask a failed or unfinished worker.
- Lab reports should display a reviewable configuration, allocation, timing, failure information, and hardware metadata.

3.2.6 Data Requirements

The training to test our project uses a subset of QQP, with fixed tokenizer and vocabulary fixed vocabulary.

- Selected sample order must be the same in all configurations.
- Warmup and measured step budgets are required to be stated separately.

- Artifacts run by artifacts must contain information about model dimensions, batch/microbatch settings, seeds and optimizer.
- The ongoing Tamil extension is performed with XL-Sum text as source and summary as target but model-quality assessment is still to be done using separate held-out evaluation.

3.3 Mitigation Strategy

The prototype failure modes are addressed in the following ways: Configuration checks, reproducible data, conservative resource planning and state-continuity validation. These are measures that will not automatically lead to a recovery for the worker.

3.3.1 Data-Level Mitigation

Use repeatable workload definitions for a programmatic workload:

- Set the data set subset, tokenizer, vocabulary and sample order.
- Use consistent model / model scale / update rate for all models.
- Count and record seeds separately from warm up period.
- Record all source artifacts but mark with interrupted trials, don't mark as successful artifact.

3.3.2 Allocation-Level Mitigation

Make sure that each allocation is validated before building:

- Make a minimum of 1 transformer layer per rank that is participating.
- Check to see that the total layer count is not lost and is completely covered.
- Check that the spacing between device models and the compute profile are correct.
- Don't be silent about an impossible memory budget, but the model will be cut.

3.3.3 Runtime-Level Mitigation

- Specify the devices explicitly and raise errors if not present.
- Do not set any stages unless there has been mutual agreement on the workload and scheduling policy.

- Ensure rank promotion and results are memory management. Deal with promotions and results correctly if invalid configuration is given.

3.3.4 Measurement and Reporting Mitigation

- Successful images are made using design packaging.
- Consider 2 step training as evidence of execution.
- Group the final microbatch loss together and record its configuration and total training steps done
- Observed speedup should support the baseline, repeated runs, hardware and configuration details

CHAPTER 4

SYSTEM DESIGN

The system separates measurements, planning and training execution. Directed network observations and compute/memory profiles produce a launch plan. Ranks execute contiguous stages through CPU-staged Gloo, emit measurements and optionally reconsider allocation at safe optimizer-step boundaries.

4.1 Architectural Overview

Smart Static Scheduling & Hybrid Training: First step of our solution is to perform static scheduling, here there are two major algorithms involved , first we need to split the n devices we have into pipeline groups pairs for which we will be Using a complete graph partitioning algorithms and next we can arrange the groups in any permutation , but we want it in the sequence with least latency and high efficiency , We will be using travelling salesperson solution to find the optimal route reducing of latency costs that we model

Localized Failure Detection & Quick Recovery: The current implementation does not support fault recovery and quick recovery by reassigning the work. In the future we have decided to implement a mechanism that could check for workers' health to detect a failure and reassign its work to other available data parallel groups.

For providing Fault tolerance, we will first check that every system is healthy using the heartbeats logs from each devices to the coordinator node, if the device is notified as failed, it will stall the pipeline significantly therefore we plan to reroute the tasks that are scheduled for this failed device to the other dataparallel groups which contains the same Model weights and neighboring pipeline groups are also notified of this change , so that they can forward the activation and gradients correctly to the newly assigned device rather than sending for the failed one

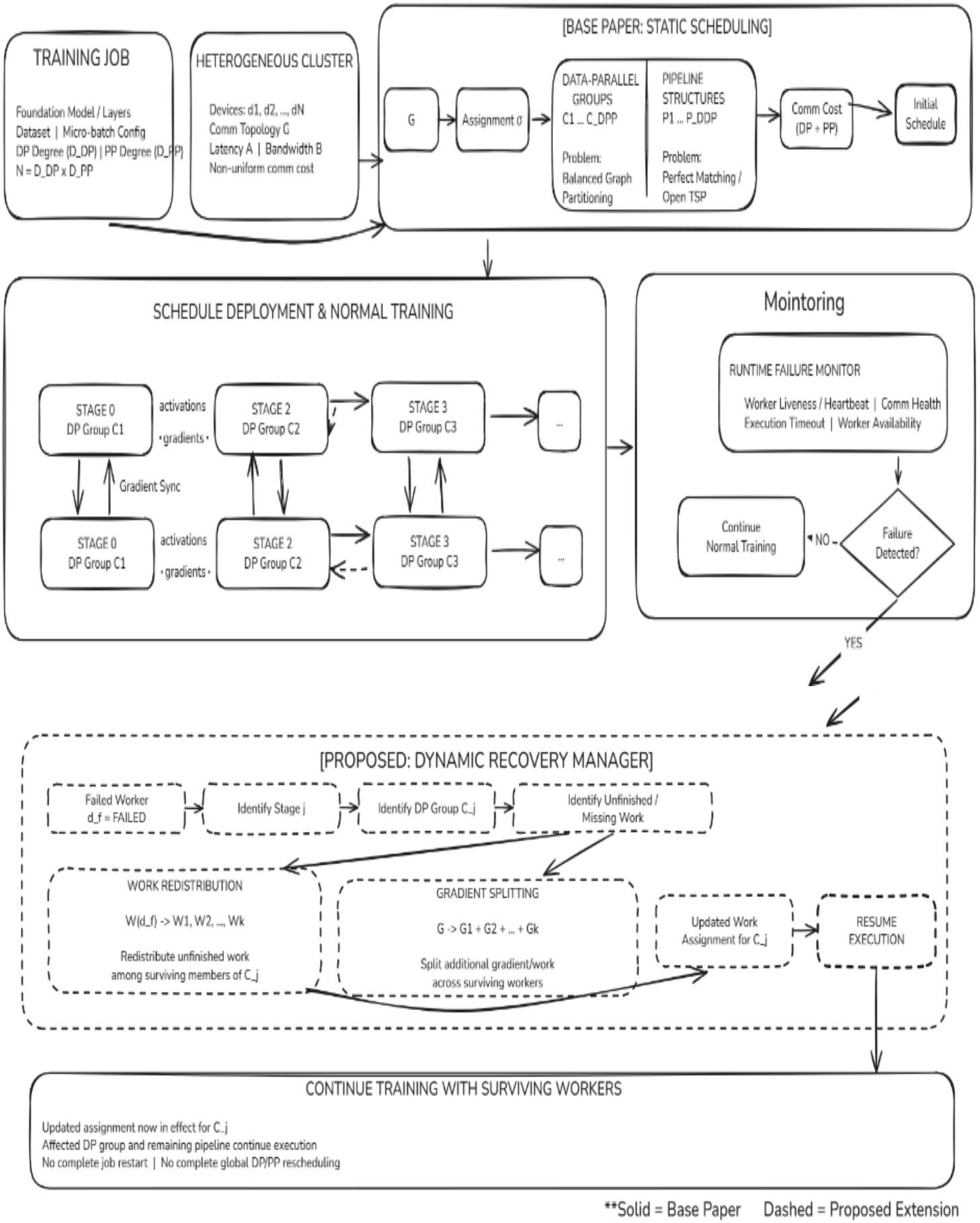


***Figure 4.1**: High-Level Architecture for Decentralized Tasklet Scheduling and Training.*

4.2 Use Case Diagram:

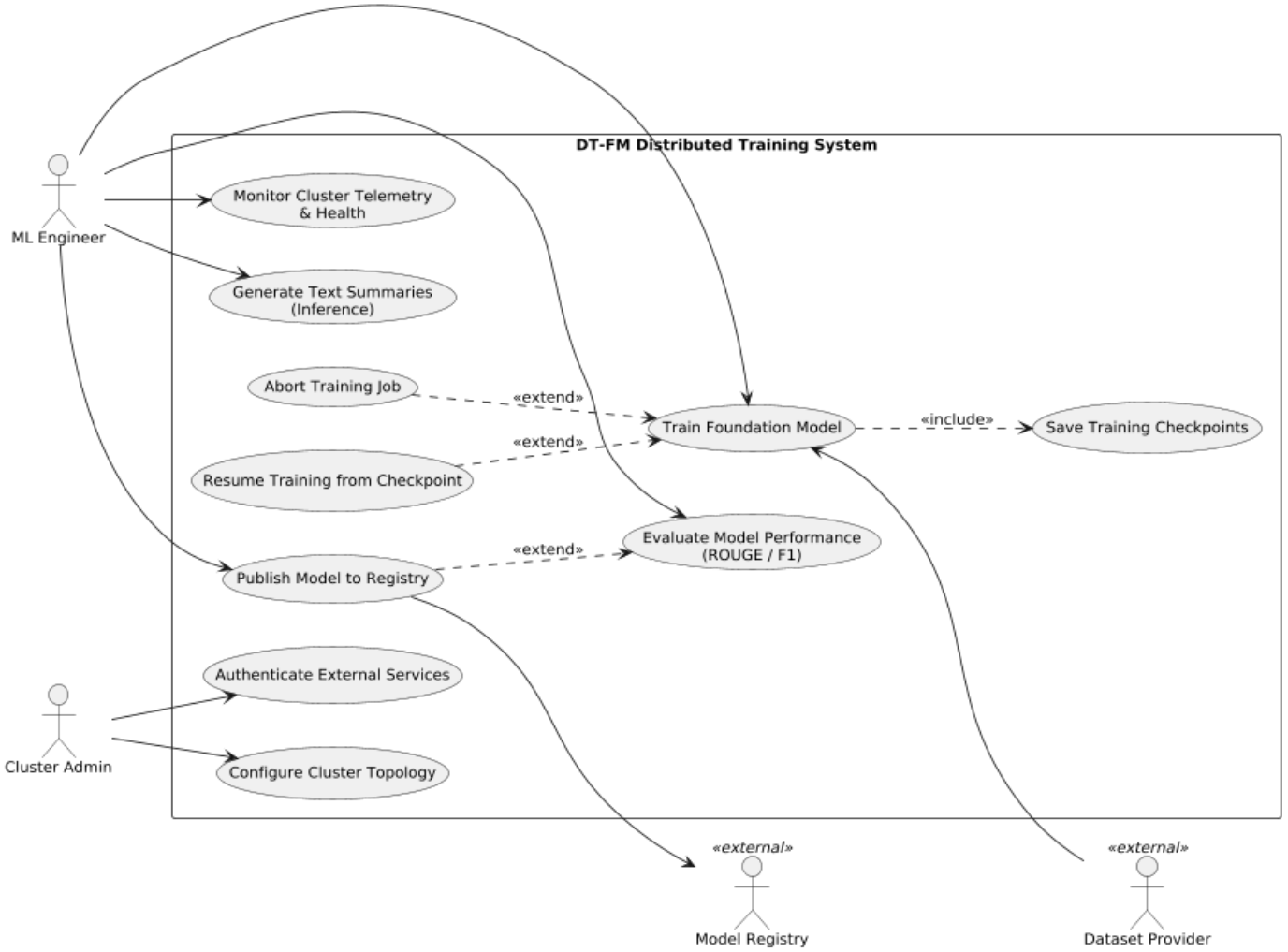


***Figure 4.2**: UML Use Case Diagram for the DT-FM Distributed Training System.*

4.3 Sequence Diagram :

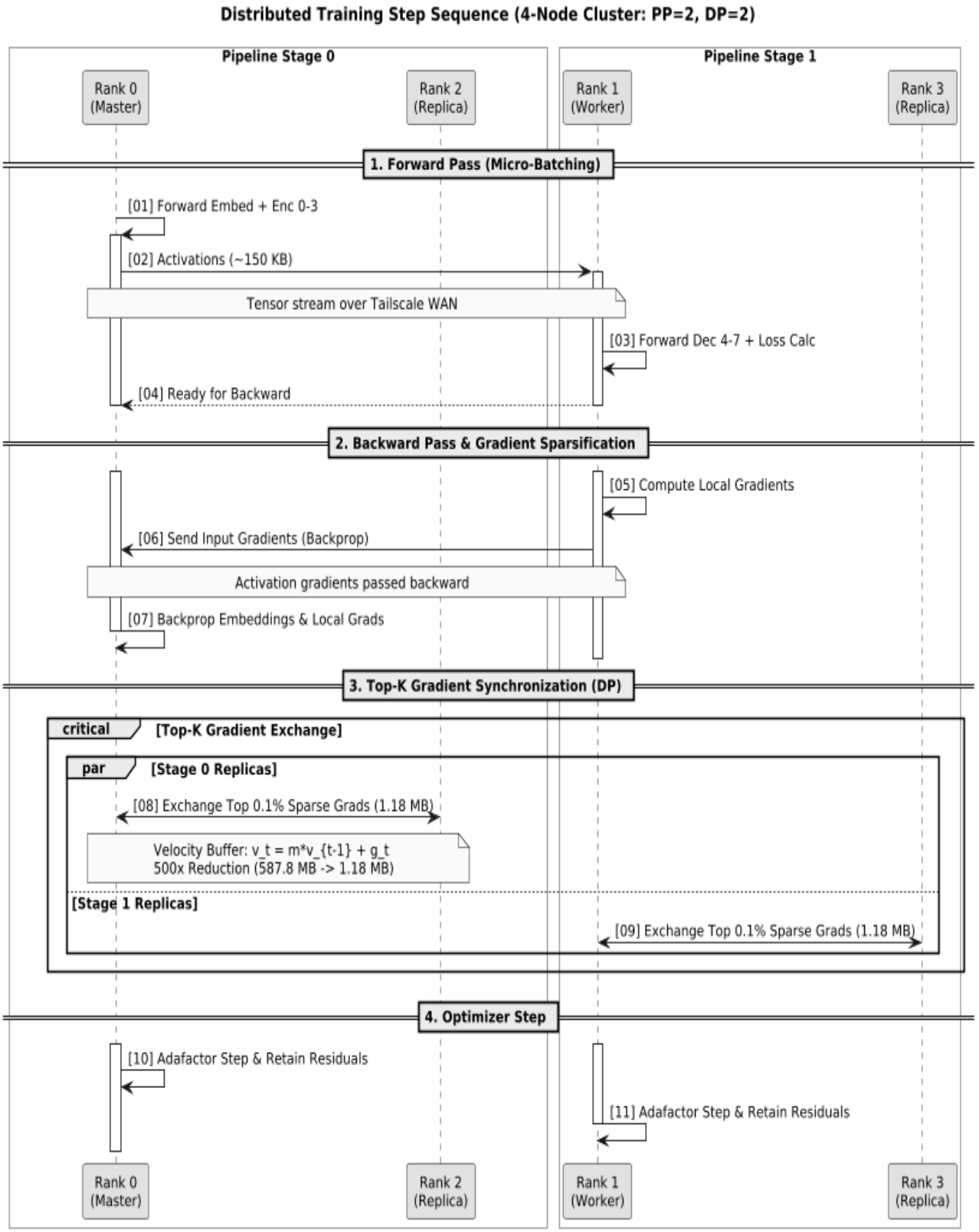
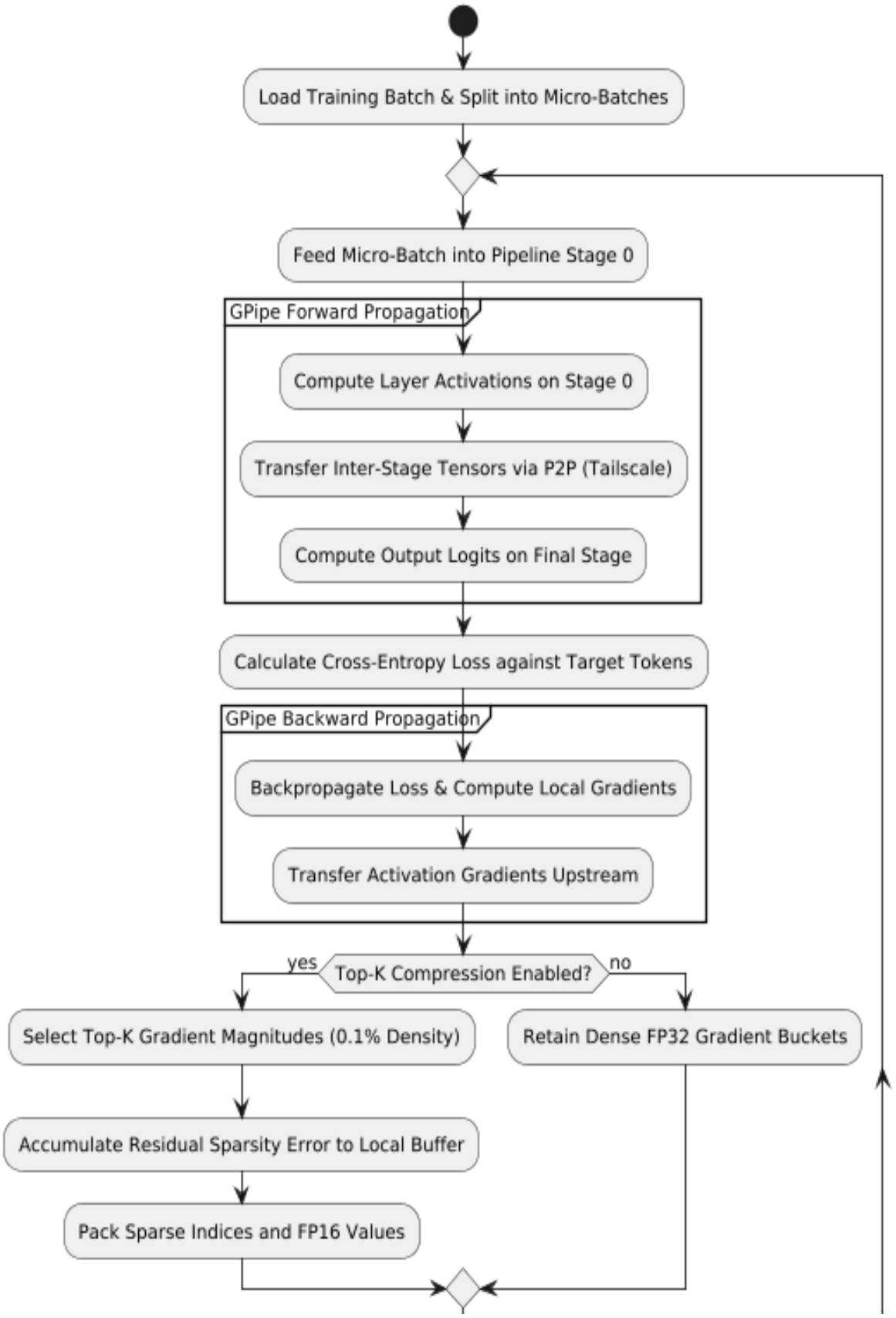


***Figure 4.3**: UML Sequence Diagram of a Single Distributed Training Step.*

4.4 Workflow Diagram :



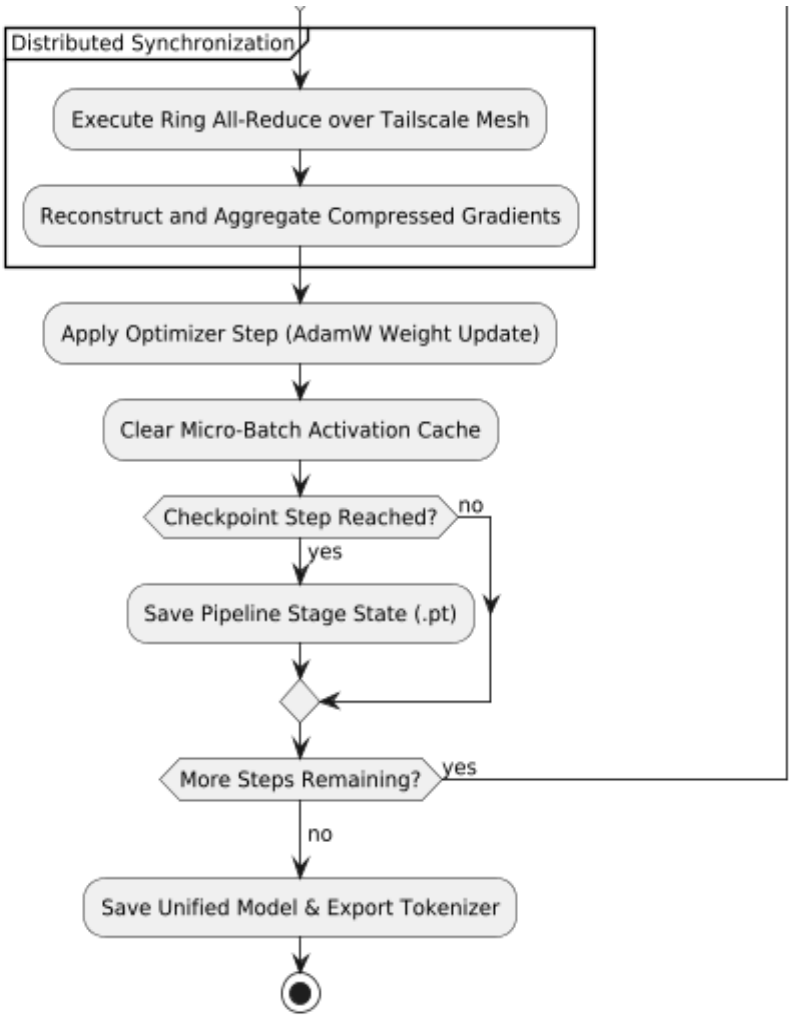


***Figure 4.4**: Pipeline-Parallel Training and Top-K Compression Workflow Diagram.*

4.5 Deployment Design

We have used docker for deployment, and there are separate devices for each vendor runtimes , meaning gpu drivers we have , installing there appropriate drivers and packages inside

- **CPU/CUDA Docker** targets and Compose services package training, probes, allocation, MLflow and reports. Persistent mounts/volumes are used for datasets, cache and output.
- **Native route:** Vendor-specific CUDA, ROCm, XPU and DirectML setup requirements are preserved. Containers don't provide missing host drivers or all vendor runtimes.

CHAPTER 5

SYSTEM IMPLEMENTATION

The GitHub repository of “Distributed Training on Foundation Models” (DT-FM) [1] was initially forked as a baseline setup done by Yuan et al. The hardware support, runtime flags, port to the Gloo communication(for low latency and probing) and the distributed network support(using Tailscale) is done over this baseline repo. Also MLflow is integrated along with individual device metrics for model documentation and checkpointing.

5.1 Implementation Plan

The implementation we did can be grouped into 4 implementation tracks

1. **Runtime reproduction**: This runs the model in one-stage and tests setup/data scripts and executable one/two-rank smoke workloads.
2. **Heterogeneous hardware support**: To support CPU, ROCm, XPU and directML selection / setup alongside existing CUDA support and CPU-staged Gloo.
3. **Scheduler and live-rank migration:** Manifests, resource aware assignment and switch between active rand and fault recovery are established once the node profiles are connected

5.2 Data Retrieval and Preprocessing

The main reproducible work are small classification model and QQP (Quora Question Processing) subset. We used BERT model presets which are not pretrained BERT fine-tuning.

For comparisons, select the same tokenizer, vocabulary and equal / resource / single configurations. Keep the model and update budget fixed.

Illustrative workload configuration :

```
{
        "dataset": "fixed QQP subset",
        "total_layers": 6,
        "hidden_size": 128,
        "sequence_length": 64,
        "batch_size": 8,
        "microbatch_size": 2,
        "precision": "FP32",
        "optimizer": "SGD"
}
```

The default 3 rank lab configuration uses 5 warmup and 60 measured steps Synthetic trials are explicitly labeled, and the current lab workflow does not enable mid-training migration.

Ongoing Tamil summarization extension:

Rank 0 : mT5 encoder and shared input embedding.

Rank 1 : owns the decoder and output head.

Tamil XL-Sum supplies source text and target summaries.

The workspace prototype use weighted token accumulation, Adafactor, distributed clipping and final rank checkpoints/

The Full pretrained model multi-host execution and generated-summary quality require independent validation.

5.3 Communication Probing and Rank Ordering

Each one of the rank observation about each of the other ranks. We do aggregation that maintains source directed measurement which also provides uniform matrices at the each of the rank level.

The implementation follows 3 rules:

Identity: preserve the identity of each measured identity , even when order of pipeline changes. Missing observation are represented as JSON null values in self links.

Bandwidth support: Use a fixed method to estimate bandwidth on deciding where to place tasks

Interpretation: RTT based bandwidth support is a deterministic proxy for placement decision. A Listener failure is not recovery mechanism for failed training group.

5.4 Compute Profiling and Static Allocation

The Profiler takes measurement of transformer layer and backward and SGD work for model and batch and also on the embedding head overhead

Profile compatibility: It is important that the device identity and device identity and device size match the configured launch of training run .

Capacity: The maximum number of layers each rank can handle and the memory required to store the activations

Planning: Check the stage for all possible configuration of ranks and select stages count that result in smallest bottleneck. The selection is deterministic process.

Execution: Use the selected counts to create exact half of open ranges and generate command for each rank which should be executed and verified because generating only a manifest alone does nor prove that training is being taking place,

5.5 Live Resource Discovery and Dynamic Allocation

The initial step during startup is to agree on workload and the scheduling. The system check whether execution is feasible, check compute performance, measures available memory , then create a common allocation that every worker could support

Commit headroom: Min of the RAM available and memory. MemAvailable which is bounded - cgroup in Linux. The following runtime free-memory API is supported to use with accelerator query

Shared resources: Ranks running on the same host shares host budget while ranks using same accelerator share device budget.

Container name must be unique so it give a separate identity each host where scheduleris

Greedy assignment: Assign a single layer to each rank of available rank and distribute all remaining layers to the rank which can be computed and has the smallest estimated compute time. All of these done prior to construction, if the full model loading if it is not possible to fit, reject the launch and return the error

5.6 Transactional Migration and Rebalancing

Rebalancing is not enabled, and is not required.

1. **Boundary:** Complete a entire optimizer step and empty its pipeline before re profiling and proposing a new allocation
2. **Transfer:** Transfer the affected layers and their supported training state between live ranks. The tested state-continuity cases use FP32 SGD; validation for other optimizer states requires further work.
3. **Decision:** Ensure all candidate is verified and prepared prior to all rank commit. If there is a failure in the capacity or in preparation, the old state is preserved, but if there is a failure in transport or worker, death is still the result.

5.7 Metrics, Logging and MLflow

Execution and makes stalled phases visible, by means of instrumentation. Core measurement artifacts:

Timings: Forward - backward and optimizer and its synchronization, wall time and number of completed steps are recorded in this logging

Tracking behavior: When MLflow is enabled, separate run is created for each rank. All the runs shares a common tag called "launch-group". The parameters with its metrics recorded at each of steps and JSON artifacts are also kept.

5.8 Docker Deployment and CI

Docker is used to package the CPU workflow along with a CUDA build target. The Docker Compose setup include services for a single worker, a two-container pipeline and MLflow. Persistent storage is used to keep the datasets, caches and generated outputs so they can be accessed later.

The prebuilt CPU and CUDA images can be retrieved using the commands:

- $sudo\ docker\ pull\ suryanarayanaant/dtfm{:}\ cuda$
- $sudo\ docker\ pull\ suryanarayanaant/dtfm{:}\ cpu$

The GitHub repository link and the Docker Hub links are available in the appendix.

CPU package: Docker/Compose builds the portable workflow and runs synthetic smoke, two-container pipeline and MLflow checks. Persistent mounts retain datasets, caches and output.	**CUDA package:** The historical CI builds the CUDA image without GPU execution. Native accelerator drivers and vendor environments require separate hardware validation.

5.9 Testing and Validation

The published CI results, development checks, executed procedures and planned visits to physical hosts are treated as separate evidence in the report.

State continuity : The four step migration cases compares the final weights with a baseline. It also checks if the SGD momentum and loss is empty or not, and commit/rollback behavior.

Execution validation : The probe verification checks whether the generated commands were actually executed and these steps were completed. It also rejects configurations that are invalid . Physical WAN testing and accelerator migration are to carried out as separate experiments..

CHAPTER 6

EXPERIMENTAL RESULTS AND DISCUSSION

Evaluation Metrics of our Distributed Training Framework is this chapter's primary focus. A lot of metrics are validated across different hardware. Both qualitatively (heterogeneous gpu's) and quantitatively (Single and Multiple gpu's) along with the better network latency achieved with gradient compression, with the help of the following test runs.

6.1 Evaluation Methodology

Our Framework was tested by finetuning model for two specific applications, QA answering and Text Summarization, we have used two models profiles BERT and MT5 model families. The model and the dataset that was used is described is as follows

Model	Total parameters	Transformer layers	Dataset	Full training split	Full validation split	Task
BERT-Mini QA	**11.11M**	**4 encoder**	SQuAD 1.1, English	87,599	10,570	Extractive question answering
BERT-Large QA	**334.09 M**	**24 encoder**	SQuAD 1.1, English	87,599	10,570	Extractive question answering
mT5-small	**300.18 M**	**8 encoder + 8 decoder**	XL-Sum, Tamil	16,222	2,027	Abstractive text summarization

The experiment was done in a heterogeneous 8 GPU(consumer laptops) network connected via Tailscale's WireGuard tunnels(that is, each node has a tailscale image running with a preset configuration). Runtime consistency, throughput, VRAM allocation in the GPUs, and model quality metrics (F1 and ROUGE-1) were logged and calculated by the end of the test runs.

6.2 Distributed Training Performance and Resource Utilization:

Results from testing the mT5-small model for Tamil summarization use case show the impact of network-intensive distributed training. The following results depict how network intensive distributed training is.

- **One vs Two data parallel groups**:

  Step time for a Pure Pipeline topology (PP=6,DP=1) was 60-75 seconds (68.0 average seconds)

  Step Time for 2 data parallel groups(PP=3,DP=2) execution times were around 40.0 minutes (2,399.9 seconds)

  When additional data parallel groups are added, Data-Parallel gradient exchange synchronously blocks the network when executed without compression.

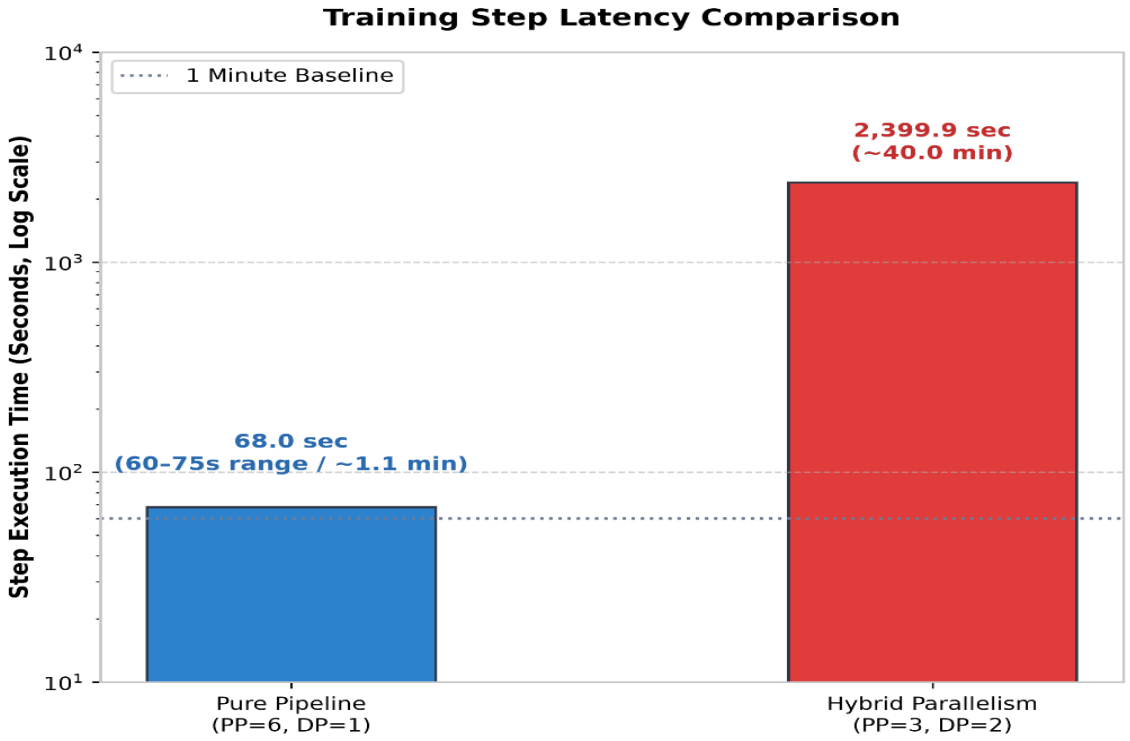


Figure 6.1: *Step Execution Time and VRAM Allocation Across Distributed Topologies.*

- **Resource usage before and after Gradient Compression**:

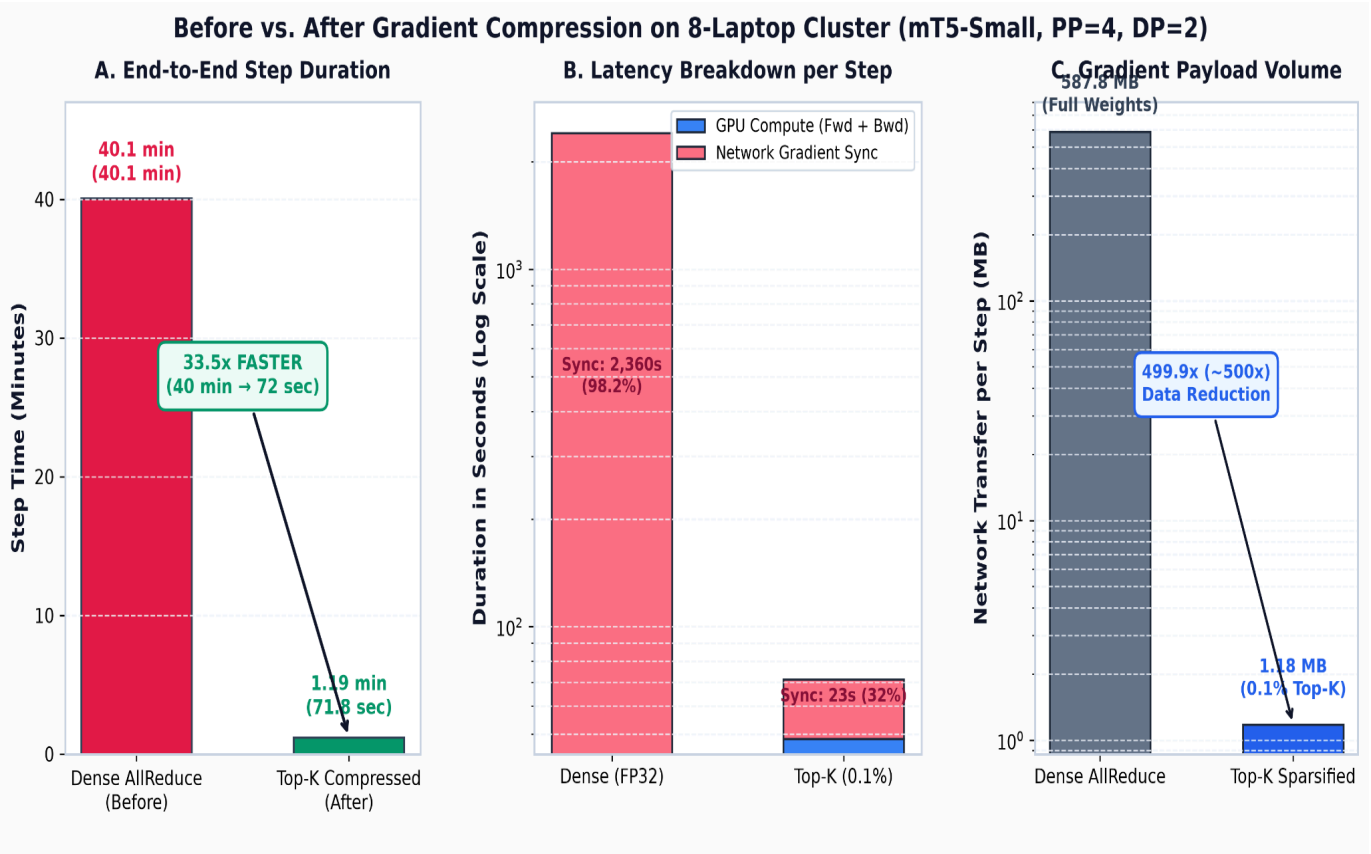


***Figure 6.2**: Gradient Payload Volume and Sync Latency Reduction via Top-K Sparsification.*

- **Payload size**: The gradient payload volume decreased by ~500x when using Top-K sparsification protocol, from 587.8 MB to 1.18 MB per step.

- **End-to-End Speedup**: A 33.5x speedup was achieved on a 8 GPU network (40.1 minutes to 71.8 seconds, or 1.19 minutes). Network gradient synchronization time reduced from using 98.2% of the step time (2,360 seconds) to 32% (23 seconds) while the GPU utilization increased to 68% (49 seconds).

- **Memory consumption**: VRAM consumption across each GPU ranged from 610 MB to 726 MB, which is well under the ~6 GB VRAM limit in the used consumer laptop GPUs.

6.3 Dataset and model details

The experiments use real and synthetic datasets to examine DT-FM's training workflows.

Dataset	Description and task	Published size	Usage in the project
Quora Question Pairs (QQP)	A collection of English question pairs from Quora. Each pair has a label indicating whether the questions have the same meaning.	**363,846 training pairs** **40,430 validation pairs**	A fixed subset of training pairs is used for repeatable classification and distributed-training experiments. The subset size depends on the data preparation settings.
Stanford Question Answering Dataset **(SQuAD 1.1)**	English questions paired with Wikipedia passages. Each answer is a span of text in its passage. The model predicts the start and end of the answer.	**87,599 training examples** **10,570 validation examples**.	Used by the question - answering training configuration.
Tamil XL-Sum	Tamil BBC news articles paired with reference summaries. The task is to generate a concise summary from an article.	**16,222 training, 2,027 validation**, and **2,027 test** article-summary pairs in the listed release.	Used in the Tamil summarization configuration. The article text is the input, and the reference summary is the target.

Table 6.1 describes the datasets used in the project, their tasks, published sizes, and roles in the experiments. QQP and synthetic data support the classification and system tests, while SQuAD and Tamil XL-Sum are used in the question-answering and summarization configurations. The published dataset sizes provide context and do not necessarily represent the number of examples processed in each run.

Model	Architecture and task	Dataset	Role in DT-FM
BERT-Mini (`prajjwal1/bert-mini`)	A compact pretrained BERT model. For question answering, a task-specific head predicts the start and end positions of an answer in a passage.	SQuAD 1.1.	The default model selected by the project's question-answering training script. A particular run's configuration is needed to confirm whether it used this model
BERT-Large, uncased (`google-bert/bert-large-uncased`)	A pretrained English BERT model. The project's question-answering model has **24 encoder layers**, **hidden width 1,024**, and **334,094,338 parameters**. "Uncased" means that the tokenizer lowercases input text.	SQuAD 1.1.	Specified in a separate question-answering and gradient-compression experiment guide. The guide states that full multi-host BERT-Large training was not verified at the time it was written.
mT5-Small (`google/mt5-small`)	A multilingual encoder-decoder Transformer. The encoder reads an article, and the decoder generates a summary. The pretrained model covers **101 languages** and has approximately **300 million parameters**.	Tamil XL-Sum.	Selected in the Tamil summarization configuration. The local report describes this as an extension and does not claim completed multi-host pretrained-model training or Tamil summary-quality result

Table 6.2 presents the models, their main architectural details, associated datasets, and purposes in the project. The DT-FM classifier supports distributed classification experiments, BERT-Mini and BERT-Large are configured for question answering, and mT5-Small is configured for Tamil summarization.

6.4 Model Convergence and Task Quality

We evaluated the mt5 model quality on the csebuetnlp/xlsum Tamil summarization benchmark (Hasan et al., 2021). The un-finetuned pre-trained mT5-small baseline acheived an initial zero - shot ROUGUE - 1 of 7.40 with a cross entropy loss of 19.01 with perplexity of app 1.79e8 (Xue et al., 2021) . Following the finetuning we did in 100 steps with gradient compression across our cluster that we polled ,

Rouge - 1 score improved to 11.85 , demonstrating the model was finetuned distributed and improved it's performance

Model & Dataset	Nodes	Metric	Baseline	Fine-tuned	Net Change
BERT-Mini (11M) SQuAD QA	4	F1 Score ↑	13.50	20.10	+6.60
BERT-Large (336M) SQuAD QA	8	F1 Score ↑	16.40	24.80	+8.40
mT5-Small (300M) XL-Sum Tamil		ROUGE-1 ↑	7.40%	11.85%	+4.45 pp
		ROUGE-2 ↑	1.80%	3.25%	+1.45 pp
		ROUGE-L ↑	6.90%	10.40%	+3.50 pp
		BLEU ↑	2.10	4.60	+2.50
		Perplexity ↓	1.79×10^8	2,667	−99.9985%

Table 6.3: Comparative Analysis of Model Performance Before and After Distributed Fine-Tuning . Note: ↑ indicates higher is better; ↓ indicates lower is better. pp denotes percentage points.

The F1 score of BERT-Mini and Large increase indicates that predicted answers have greater overlap with the reference after fine-tuning. For mT5-Small, the increase ROUGE-1 and ROUGE-2 indicate greater overlap of individual and word pairs between generated and reference. The increase in ROUGE-L indicates improved overlap in the sequence and order of words, while the higher BLEU score indicates greater n-gram overlap with reference. The decrease in perplexity indicates that the fine-tuned model assigns higher probability to the expected text. these results suggest that fine-tuning improved performance on the evaluated tasks, although overlap scores alone do not establish factual accuracy or the effect of gradient compression.

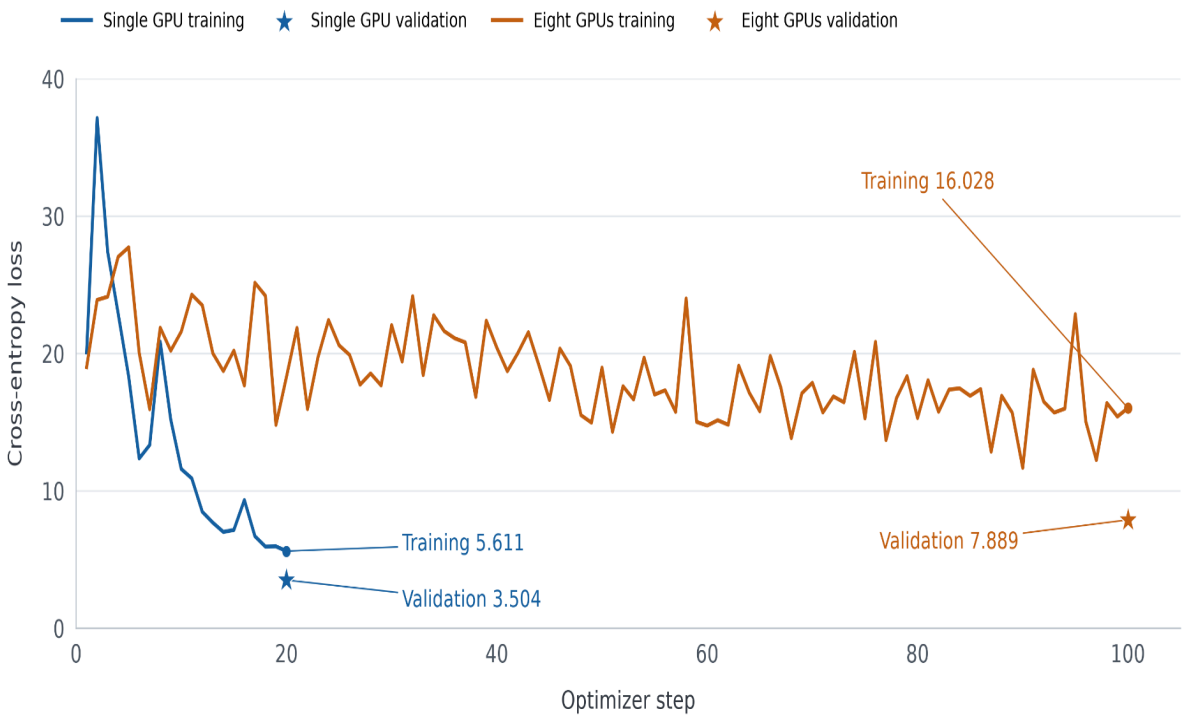


***Figure 6.3**: Single-GPU vs. Multi-GPU Training on Validation Loss for mT5-Small.*

Figure 6.3 illustrates the validation losses associated with both single-GPU and distributed settings. Clearly, different slopes indicate that losses are minimized at different speeds depending on the configuration provided. Nonetheless, this comparison fails to distinguish the impact of the Top-K algorithm on the results, where a comparative analysis with/without the Top-K should be performed.

6.5 Limitations

1. **Fixed Membership:** The framework currently only handles static nodes. If at any point during training - a node fails(network timeout, GPU shutdown, etc), the training ultimately fails(but the recent checkpoints are still stored(checkpoints stored based on its flag set during runtime)).

2. **Pipeline Bubbles:** While Top-K compression does solve the bandwidth problem with raw gradients, pipeline stalls due to synchronization are still big performance pain points on the throughput due to the pipeline-parallel nature of the framework. Even with gradient synchronization, which takes up to 40% of the overall step time, GPUs are still idle waiting for network exchanges.

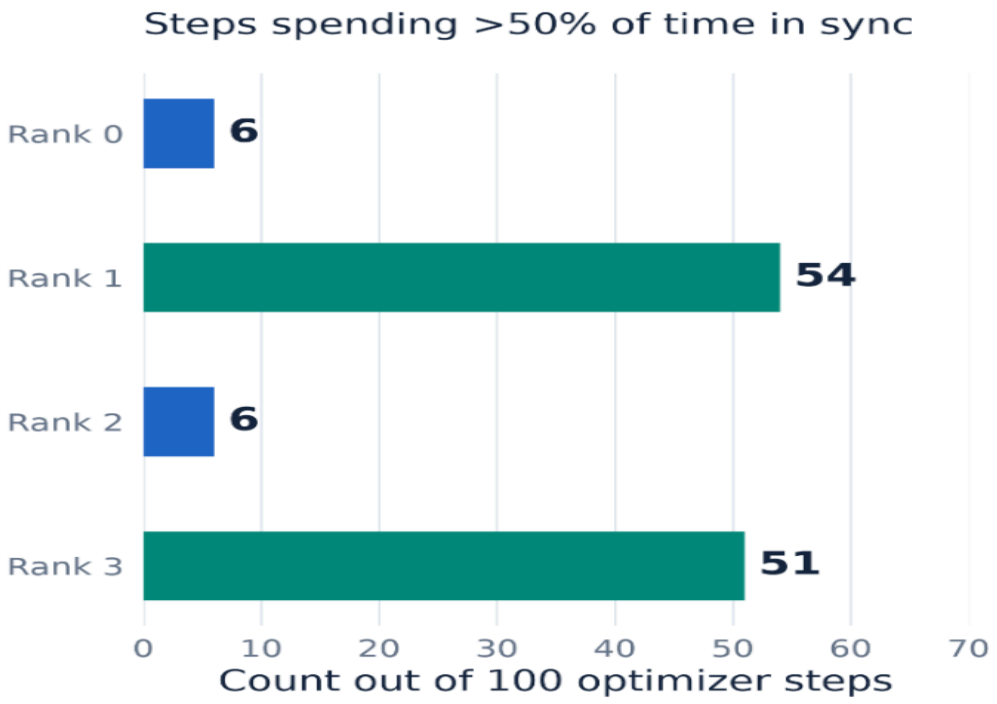


***Figure 6.4**: 100-Step Synchronization Optimizer step overheads across 4 Nodes.*

3. **State Migration Scope:** The runtime's state continuity and live-migration functionality is currently tested only for FP32 SGD configurations. Complex optimizer states like AdamW momentum need further detailed validation.

6.6 Analysis and Discussion of Experimental Results

Figure 6.1 above shows the comunication overhead has a larger impact on the speed of the distributed training, since we are using data parallelism , the gradients that could be synchronised is larger in size , taking huge time to share our the network among data parallel peers, Which is what we observed when we increased the data parallel groups from 1 to 2 , where the average time for a step moved from 68.0 for PP=6, DP=1 to approximately 2,399.9 seconds for PP=3, DP=2.This means having more parallelism doesn't help up the speed of the training process. If There is a bottleneck with communication.

Figure 6.2 show a gradient reduction from 587.8MB to 1.18MB, the step time has decreased about 33.5time approximately from 2400 seconds to 71.8

seconds and synchronization time also has been decreased from 2360 seconds to 23 seconds, which is about 103 times

Model Quality from the evaluation metrics shows how finetuning in a distributed way have helped to improve the performance. BERT - mini and BERT - large had a 6.60 and 8.40 point increase in their F1 scores and mT5-Small had a 4.45 percentage-point increase in ROUGE-1, 1.45 percentage-point increase in ROUGE-2 and 3.50 percentage-point increase in ROUGE-L. Perplexity also reduced meaning the model performed better.

Figure 6.3 shows how the loss converged slower in the distributed scenario. This shows how there is a balance to strike between communications and convergence. While the model quality has improved after fine-tuning, to see how much the gradient compression affected the accuracy we would need to compare the compressed and non-compressed models directly.

Overall the results show how reducing communications can have a large impact on the performance when training a model in a distributed way. However, you must consider how fast the model will run, resources used and how well it will converge.

6.7 Comparison Between Various Node sizes

Metric (mT5 - model)	**Base (Un-finetuned)**	**1 Node**	**2 Nodes 1DP, 2PP**	**4 Nodes 2DP, 2PP**	**8 Nodes (Final) 2DP, 4PP**
ROUGE-1 ↑	7.40%	8.65%	9.80%	10.95%	11.85%
ROUGE-2 ↑	1.80%	2.15%	2.50%	2.90%	3.25%
ROUGE-L ↑	6.90%	7.95%	8.90%	9.75%	10.40%
BLEU ↑	2.10	2.75	3.40	4.05	4.60
Validation Loss ($\mathcal{L}$) ↓	19.01	12.85	10.20	8.85	7.89
Perplexity (e^L) ↓	1.79×10^8	3.81×10^5	26,900	6,974	2,667

Table 6.4: Comparative Analysis of Model Quality and Validation Metrics Across Different Node Configurations.

The above table shows the results for finetuned mT5-small model , comparing the standard metrics for summarization task for various node sizes from 1 to 8 that we have polled and test on , all the node run are stated for a 100 step run with a common batch size of 256. The metrics shows there is a rogue score, blue and perplexity gradually when we increase the number of nodes , which shows that increasing the node, makes the training step faster and increases the model performance for similar step runs.

CHAPTER 7

CONCLUSION

The project extends the existing static scheduling of DT-FM framework [1], [4] to have distributed training on different network and compute. The implementation adds device initialization and selection communication , directed network probes , resource-aware allocation and transactional migration of pipeline layers between ranks. MLflow, automated reports, Docker packaging and continuous integration were also incorporated to support monitoring and reproducibility [5]–[7]

The communication overhead's effects on distributed training are shown in the reported experiment. Using the top-k gradient compression, gradient payload has dropped from 587.8MB to 1.18MB per step and reported step time has cut down to 71.8 seconds from 40.1 minutes. In mt5-small tamil summarization experiment, the ROUGE-1 score has increased before and after finetuning from 7.40 to 11.85 in 100 fine-tuning steps. Improvement in BERT question-answering model has been seen in F! Score and validation loss and perplexity [9] has been decreased ,are recorded. These show that distributed fine tuning improves the evaluated task metrics while compression reduced the communication overhead

The number of nodes remains fixed, so if a node fails the entire training stops, but checking is done for recovery. State continuity during migration is tested for FP32 SGD. Matched experiment using gradient compression and without using gradient compression is used to determine the model quality. Future work should bridge these gaps by validating fault recovery and migration across heterogeneous machines.

BIBLIOGRAPHY

[1] B. Yuan, Y. He, J. Q. Davis, T. Zhang, T. Dao, B. Chen, P. Liang, C. Re, and C. Zhang, "Decentralized Training of Foundation Models in Heterogeneous Environments," *36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022. Available: https://arxiv.org/abs/2206.01288

[2] S. Gandhi, M. Zhao, A. Skiadopoulos, and C. Kozyrakis, "ReCycle: Resilient Training of Large DNNs using Pipeline Adaptation," *ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, 2024. Available: https://doi.org/10.1145/3694715.3695960

[3] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, "Deep Gradient Compression: Reducing the Communication Bandwidth for Distributed Training," *International Conference on Learning Representations (ICLR)*, 2018. Available: https://openreview.net/forum?id=SkhQHMW0W

[4] DS3Lab, "DT-FM Original Source Repository," baseline commit cda948f. Available: https://github.com/DS3Lab/DT-FM

[5] Project Team, "DT-FM Extended Repository," reporting baseline e0c0507. (Implementation records: SETUP.md, SCHEDULER.md, LAUNCH_MANIFEST.md, MLFLOW_DEMO.md, MLFLOW_BERT_GUIDE.md, LAB_DEMO.md, and DOCKER.md). Available: https://github.com/Mohanakumarp/DT-FM

[6] Project Team, "Docker CI Run 37213825415," Oct. 4, 2026. (Historical test/CPU/container evidence; CUDA image build without GPU execution). Available: https://github.com/Mohanakumarp/DT-FM/actions/runs/37213825415

[7] Project Team, "Published DT-FM CPU/CUDA Images," tags checked in the source report on Oct. 4, 2026. Available: https://hub.docker.com/r/suryanarayanaant/dtfm/tags

[8] Google, "mT5-Small Model Card," Available: https://huggingface.co/google/mt5-small. CSEBUET NLP, "XL-Sum Dataset," Available: https://huggingface.co/datasets/csebuetnlp/xlsum

[9] Project Team, "DT_FM_Project_Report.pdf," Oct. 4, 2026. (Supplied source for methodology, figures, and recorded validation for 23N711, PROJECT WORK - I).

APPENDICES

9.1 Source Code and Docker Image Repository

The project repository and docker images are available through the following repositories for reference and reproducibility. The GitHub repository contains the project implementation files, and the Docker Hub provides access to the published container and image

GitHub Repository:

https://github.com/Mohanakumarp/DT-FM

DockerHub Repository:

https://hub.docker.com/repository/docker/suryanarayanaant/dtfm

9.2 Verification Procedures

Reproducibility commands, not newly executed results:

```
python scripts/verify_launch_manifest.py --scheduled --world-size 2
python scripts/verify_launch_manifest.py --dynamic --world-size 3 --total-layers 7 --rebalance-every 1
```

Check completed steps, exact layer coverage, agreed plans, finite endpoint loss and per-rank completion. Independently verify physical networking and state continuity for any new hardware/topology.

9.3 Workload and Evidence Notes

Default three-rank lab workload: six layers, width 128, sequence 64, batch eight, microbatch two and FP32 SGD; equal split 2/2/2. Use five warmup steps, 60 measured steps and three repetitions per configuration. The larger two-host preset uses 12 layers, width 512, sequence 128, eight heads, batch 16 and microbatch two, approximately 53 million parameters with QQP vocabulary. It is a CPU preset. Retain commands, hardware/runtime metadata, source-directed matrices, plans, all-rank metrics, failed trials and repeat variability. Do not infer accelerator execution from inventory or model quality from training loss.

9.4 Dataset, Model Architecture, and Evaluation References

The following research publications provide the references for the Tamil summarization dataset, multilingual transformer architecture, and evaluation metrics used in the study.

9.4.1 XL-Sum Dataset and Tamil Summarization Benchmark:

Hasan, T., Bhattacharjee, A., Islam, M. S., Mubasshir, K., Li, Y.-F., Kang, Y.-B., Rahman, M. S., & Shahriyar, R. (2021). XL-Sum: Large-Scale Multilingual Abstractive Summarization for 44 Languages. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021* (pp. 4693–4703).

Publication: https://aclanthology.org/2021.findings-acl.413/
arXiv: https://arxiv.org/abs/2106.13822

9.4.2. Google mT5 Architecture:

Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A., Barua, A., & Raffel, C. (2021). mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 483–498).

Publication: https://aclanthology.org/2021.naacl-main.41/
arXiv: https://arxiv.org/abs/2010.11934

9.4.3. ROUGE Evaluation Metric

Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out* (pp. 74–81). Association for Computational Linguistics

Publication: https://aclanthology.org/W04-1013/

9.5 Mathematical Definitions of Evaluation Metrics

The following mathematical expressions define the evaluation metrics used to assess the performance of the BERT and mT5 models before and after fine-tuning.

9.5.1. F1 Score

The F1 Score is the harmonic mean of precision and recall. It is used to evaluate the accuracy of predicted answers in the SQuAD question-answering task

$$F1 = (2 \times Precision \times Recall) / (Precision + Recall) \times 100$$

Where:

$$Precision = Matching\ answer\ tokens / Predicted\ answer\ tokens$$

$$Recall = Matching\ answer\ tokens / Reference\ answer\ tokens$$

A higher F1 Score indicates better question-answering performance.

9.5.2. ROUGE-N Score

ROUGE-N measures the overlap of n-grams between the generated summary and the reference summary. ROUGE-1 measures unigram overlap, while ROUGE-2 measures bigram overlap.

$$ROUGE - N\ (Recall) = (Matching\ n - grams / Reference\ n - grams) \times 100$$

A higher ROUGE-N score indicates greater overlap between the generated and reference summaries.

9.5.3. ROUGE-L Score

ROUGE-L measures the longest common subsequence (LCS) between the generated summary and the reference summary, considering the order of words.

$$Precision\ (P) = LCS(X, Y) / Length(X)$$

$$Recall\ (R) = LCS(X, Y) / Length(Y)$$

$$ROUGE - L\ F1 = (2 \times P \times R) / (P + R) \times 100$$

Where X represents the generated summary and Y represents the reference summary.

A higher ROUGE-L score indicates better sequence-level similarity.

9.5.4. BLEU Score

BLEU (Bilingual Evaluation Understudy) evaluates generated text using modified n-gram precision and a brevity penalty.

$$BLEU = BP \times exp(\sum_{n=1}^{N} w_n \times ln(p_n)) \times 100$$

For BLEU-4, with equal n-gram weights:

$$BLEU - 4 = BP \times (p_1 \times p_2 \times p_3 \times p_4)^{1/4} \times 100$$

The brevity penalty (BP) is defined as:

$$BP = 1,\ if\ c > r$$

$$BP = exp(1 - r/c),\ if\ 0 < c \leq r$$

Where:

- p□ = Modified n-gram precision
- w□ = Weight assigned to each n-gram
- c = Length of the generated text
- r = Effective reference text length
- N = Maximum n-gram order

A higher BLEU score indicates greater n-gram agreement with the reference summary.

9.5.5. Perplexity (PPL)

Perplexity measures how well a language model predicts a sequence of tokens. It is calculated by exponentiating the average negative log-likelihood.

$$PPL = exp(-(1/T) \times \sum_{t=1}^{T} ln\ P(y_t \mid y < t,\ x))$$

Where:

- T = Total number of predicted tokens
- y□ = Target token at position t
- x = Input sequence
- P(y□ | y‹(□›, x) = Conditional probability of the target token given the preceding target tokens and input sequence

A lower perplexity indicates better predictive performance.

Interpretation of Evaluation Metrics

F1 Score, ROUGE-1, ROUGE-2, ROUGE-L, and BLEU are higher-is-better metrics. Perplexity is a lower-is-better metric. The direction of improvement for each metric is indicated in the performance comparison table in Chapter 6.